PUBSUB SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wad

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

1. Your system requires guaranteed delivery of every message. How would you design the Pub/Sub subscriptions?

I'd design for at-least-once delivery as the baseline and make the whole pipeline idempotent - treat duplicates as normal. Pub/Sub natively provides durable storage and retries, so rely on that but never assume "exactly-once" without extra protections. Include a unique message id (producer-generated UUID or deterministic event key) in each message attribute so consumers can detect and ignore duplicates easily.

Make the subscription durable and resilient: use a pulled subscription (streaming pull) or push with a robust endpoint, set a sensible ack-deadline, and ensure the consumer acks only after the message has been persisted/committed to durable storage (DB/BigQuery/File). If processing can be long-running, implement ack-deadline extensions (lease management) so Pub/Sub won't redeliver while you're still safely working the message.

Add operational safety nets: configure a Dead Letter Topic (DLQ) for messages that repeatedly fail after N attempts so you never lose failing messages - they'll be quarantined for manual inspection. Also set a message retention duration long enough to cover outages or long backpressure windows so Pub/Sub keeps messages until you can recover and process them.

For end-to-end guarantees, include observability and a runbook: record message receipt and processing status in a lightweight control store (Firestore, BigQuery, or Spanner) keyed by message id so you can prove whether each message was processed. Emit metrics (publish count, ack count, DLQ count, oldest unacked age) and alert on divergence between publish and processed counts so you detect delivery gaps quickly.

Finally, test with chaos scenarios: simulate consumer failures, network partitions and long processing times to verify that redelivery, DLQ behavior, and idempotent consumers correctly preserve every message. In short: rely on Pub/Sub durability and retries, make consumers idempotent, persist processing results before ack, and use DLQs + monitoring for any problematic messages.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

2. A downstream system requires messages in strict order. How would you configure Pub/Sub for this?

Use Pub/Sub ordering keys and enable message ordering on the topic/subscription - that's the main mechanism Pub/Sub provides for per-key FIFO. Producers must set the same orderingKey attribute for messages that must stay in order (for example customerId or a single global key if you need total order), and subscribers must process and ack messages for a given ordering key in the order they arrive.

Remember the caveats: ordering is per ordering key, not global across all keys. If you need total ordering across all messages, you'll need a single ordering key (which reduces parallelism) or an external sequencer that batches/serializes messages before publishing. Design trade-off: strict global order means a single hot path and lower throughput, so only use it when business logic truly requires it.

Handle failures carefully because Pub/Sub will hold back later messages for the same ordering key until the earlier message is acked; if one message fails repeatedly it can block that key. To avoid permanent blocking, configure a Dead Letter Topic or implement a bounded retry policy and move consistently-failing messages to a DLQ so the stream for that key can continue after remediation.

On the consumer side, ensure single-threaded, ordered processing per key (or use keyed worker pools that guarantee per-key single-thread). If you use Dataflow, enable ordering-key support and make sure your worker logic preserves per-key ordering semantics (no concurrent workers handling the same key). Also make sure ack/lease semantics are robust: don't ack until processing+persisting is complete - premature ack breaks ordering guarantees if you later need to roll back.

Finally, monitor per-key lag and blocked-key metrics so you can detect stuck partitions/keys and act (inspect DLQ, replay, or fix consumer bugs). Document the ordering contract for producers and consumers so everyone understands the throughput vs ordering trade-offs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

3. A consumer is missing messages due to late acknowledgments. How would you troubleshoot?

First, confirm what "missing" means - redelivered later, dead-lettered, or never processed. Start by checking Pub/Sub and consumer logs and metrics: look at subscription backlog, oldest-unacked-message age, redelivery counts, DLQ entries, and any consumer error traces. That tells you whether messages are being redelivered (late acks) or simply dropped downstream.

Next, inspect the consumer's ack behavior and processing time. Are you processing the message for longer than the subscription's ack-deadline? If so, either increase the ack-deadline or implement automatic lease extensions (streaming-pull clients should periodically extend leases while working). If the consumer crashes or restarts before acking, check for unhandled exceptions and add defensive error handling so the message is not silently dropped.

If you see messages repeatedly redelivered and then disappear (e.g., moved to DLQ), check the subscription retry policy and DLQ configuration - maybe the message hit the retry limit. Examine the DLQ contents for failure reasons and fix the underlying processing bug or relax the retry/backoff while you remedy the data issue. Make sure your consumer does not accidentally ack on error paths before persistence - ack only after durable write.

Also verify network and scaling issues: slow network or overloaded CPU can delay processing and acking; apply flow control (limit concurrency) or autoscale consumers to match load. If you use push subscriptions, check that the push endpoint is returning non-2xx codes correctly and not ACKing or dropping messages on transient failures - push endpoints must return non-2xx to trigger retries, and Pub/Sub will stop if it sees rapid failures unless configured otherwise.

Finally, add instrumentation to prevent future blind spots: record every message id you receive and when you ack it in a control table so you can reconcile publishes vs acks; alert on growing oldest_unacked_age and on sudden jumps in redelivery counts; and run a replay test (republish a known message) to verify end-to-end behavior after fixes. These steps will find why late acks happen and give you both short-term fixes and long-term safeguards.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

4. IoT devices send thousands of events per second. How would you ingest and buffer these using Pub/Sub?

Start by putting Pub/Sub as the front-line ingestion point - each device (or an edge gateway) publishes its events to a dedicated Pub/Sub topic. Use multiple topics if you need logical separation (telemetry vs alerts). Pub/Sub scales horizontally, so it handles thousands (and much more) of events/s without you managing brokers.

Use authenticated, short-lived credentials or an edge gateway that proxies device traffic (Cloud Run or a lightweight gateway) if devices can't securely call Pub/Sub directly. The gateway can validate, coalesce, and enrich events (add device metadata) before publishing; this reduces churn and keeps producers simple while centralizing security and schema enforcement.

Configure subscriptions with adequate ack-deadline and use streaming-pull (or Dataflow) consumers that extend leases during long processing. To buffer reliably, keep retention and retry policies sane and enable a Dead Letter Topic for poison messages so bad data doesn't block the stream. Don't rely on ephemeral in-memory queues - Pub/Sub's durability is the buffer.

For downstream processing, attach scalable consumers like Dataflow streaming jobs or autoscaled consumer pools that read from Pub/Sub and do windowing, aggregation, or writes to durable stores (Bigtable, BigQuery, GCS). Tune batching and client-side publish settings (max outstanding messages, batching window) to balance throughput and latency - batching reduces RPCs and cost.

Finally, instrument and secure everything: monitor publish/ack rates, backlog, oldest-unacked age, DLQ counts and latencies, and set alerts. Use per-topic quotas or partition by keys if you see hotspots. With Pub/Sub as the durable buffer, an edge gateway for constrained devices, and autoscaled consumers, you get a resilient, cost-effective ingest layer for high-rate IoT telemetry.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

5. A business wants to combine daily batch uploads with real-time clickstream ingestion. How would you design this hybrid pipeline?

Design a common ingestion layer: batch files land in GCS and clickstream events flow into Pub/Sub. Keep raw material immutable in GCS for batch and also write a compacted copy of clickstream windows from streaming consumers into the same raw-area in GCS so both sources share a single raw zone for downstream ETL.

Process each source with the right tool: use Dataproc/Spark or scheduled Dataflow batch jobs for heavy daily transformations from GCS, and a streaming Dataflow pipeline for clickstream that does windowed aggregation, enrichment and writes compact Parquet/Avro files to GCS or directly to BigQuery. Make sure both pipelines produce curated outputs with the same schema and partitioning conventions so downstream analytics see a unified dataset.

For near-real-time needs, keep a streaming pathway to push lightweight aggregates or change-logs into BigQuery (or a serving store) for dashboards and alerts. For deeper analytics, schedule periodic compaction jobs that merge streaming micro-batches into larger files in the curated zone, preserving cost-effective query performance.

Orchestrate with Airflow/Composer: trigger batch runs on manifest arrival and trigger downstream compaction or promotion jobs after streaming windows finish. Record lineage and manifest which raw inputs were used for each curated partition so reproducibility and backfills are simple.

Finally, enforce contracts and monitoring: schema registry, validation steps, and DLQs for streaming; manifest+checksum validation for batch. Monitor end-to-end latency (publish → curated), data completeness, and duplicate rates. This hybrid approach gives low-latency answers where needed and efficient, accurate analytics for full-scope queries.

6. Your application needs to ingest API events into Pub/Sub. How would you integrate it?

Expose a secure ingestion endpoint (Cloud Run or API Gateway) that your application calls; the endpoint validates payloads and publishes messages to Pub/Sub. This decouples the app from Pub/Sub specifics, centralizes auth, and lets you enforce rate limits, schema checks, and simple enrichment before publishing.

If the application can call Pub/Sub directly, use client libraries with authenticated service accounts and batching enabled (max messages, max latency) to improve throughput and reduce RPC cost. For mobile or constrained clients, route through a lightweight proxy/gateway to manage credentials and reduce surface area.

Design messages with consistent attributes (event_id, event_time, source) and a producer-generated id to support deduplication downstream. Use attributes for routing (e.g., event_type) so subscribers can filter efficiently and avoid processing irrelevant events.

Make publish operations resilient: implement retries with exponential backoff on the client side and monitor publish error rates and message latencies. On the consumer side, persist events to durable stores before acking to ensure at-least-once semantics; use DLQs for poison messages to isolate problematic payloads.

Finally, secure and observe: apply IAM least privilege to publisher accounts, enable audit logs, and emit metrics for publish throughput, latency, and failures. With a simple gateway or client-library integration, robust message attributes, and strong monitoring, API events flow reliably into Pub/Sub for downstream processing.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. You must process streaming Pub/Sub data in real-time with aggregations every 5 minutes. How would you design the Dataflow job?

Use an event-time, fixed 5-minute window as the core: read from Pub/Sub, extract the event timestamp from the payload (not the receive time), then apply Window.into(FixedWindows.of(Duration.standardMinutes(5))). Event-time windows ensure the aggregation reflects when events happened, not when they were delivered. Keep your pipeline's watermark propagation correct so windows close on real-world time instead of delivery time.

Configure allowed lateness and triggers so you balance latency and completeness. For example, set allowedLateness to something reasonable for your domain (e.g., 10–30 minutes) so late events can still update the window. Use a trigger that emits at watermark (final) plus optional early firings for low-latency partial results (e.g., AfterWatermark.pastEndOfWindow() plus an early AfterProcessingTime.pastFirstElement().plusDelayOf(...)). Choose Accumulating mode if you want updates (retractable/updated aggregates) or Discarding if only the first result matters.

Aggregate with keyed transforms: keyBy the aggregation key, then use Combine.perKey (sum, count, custom combiner) inside the window. For high throughput, pick a combiner that is associative and commutative so the runner can pre-combine and scale horizontally. Use Combine.PerKey rather than manual GroupByKey where possible because it's more memory efficient and faster for large volumes.

Handle state, scalability and backpressure: enable autoscaling on Dataflow and tune worker parameters (max workers, machine types). If you need to keep short-term state (e.g., for deduplication or incremental aggregates), use Beam state & timers or an external keyed store (Bigtable) carefully to avoid hot keys. Monitor watermark lag, late-data ratio and CPU/memory to adjust window size, allowed lateness or worker pool.

Provide durability and downstream correctness: write aggregated results to a durable sink (BigQuery load jobs, Storage Write API, or append Parquet to GCS) and include a window id/run id and window timestamp so consumers can detect duplicates or late updates. Emit metrics (records aggregated per window, late events, updated windows) and alert on large late-data spikes so SREs can act before SLAs degrade.

Test and validate with real patterns: run with synthetic time-skewed traffic to exercise late events and out-of-order delivery, verify accumulation behavior, and check end-to-end latency from publish → aggregate → sink. That gives confidence in watermark heuristics, trigger settings and scale behavior before production traffic arrives.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

8. Your Dataflow job is producing duplicate results when consuming from Pub/Sub. How would you fix this?

First, design for at-least-once delivery and make deduplication explicit. Ensure producers include a stable, unique event id (producer-generated UUID or deterministic key + sequence). Don't rely on Pub/Sub message IDs for dedupe because they are not ideal for cross-retry idempotency - supply your own IDs in message attributes or payload.

Implement deduplication in the pipeline before writing final results. Options: (a) use Beam's built-in Distinct / RemoveDuplicates (stateful) keyed by the event id with a TTL that covers your retry window; (b) write a stateful DoFn that records seen ids in keyed state with expiration; or (c) check an external dedupe store (Bigtable/Spanner/Redis) before emitting - pick external store when dedupe state must survive pipeline restarts or be shared across workers. Make sure the dedupe TTL equals or exceeds your message retention/latency window.

If you're writing to BigQuery or another sink that supports dedupe keys, also send the event id as insertId (BigQuery streaming inserts) or use the sink's idempotency features; that gives an extra protection layer at write time. For file outputs to GCS, include deterministic filenames or manifests (content hash + window id) and write to staging + atomic rename to avoid double-writes from retries.

Audit and observability: emit metrics for "duplicate detected" and "duplicate dropped", and keep a small replayable log of duplicates (or DLQ) so you can inspect why duplicates occur (producer bug vs delayed retry). Add reconciliation jobs that compare published vs processed counts (publish count vs deduped processed count) using the message id control table so you can detect any logic gaps.

Finally, do end-to-end tests that simulate Pub/Sub redeliveries, consumer restarts and worker preemptions. Only after proving the dedupe mechanism survives worker crashes and pipeline restarts should you consider duplicates handled in production.

9. Messages are arriving late. How would you configure windowing and triggers in Dataflow to handle this?

Use event-time windowing with an appropriate allowedLateness so late arrivals are still accepted into their original window. Choose the allowed-lateness based on how late events typically arrive (e.g., 5–60 minutes). This tells the runner how long to keep window state open to accept late elements before finally garbage-collecting the window state.

Combine allowed lateness with triggers that emit intermediate/early results and also late firings. A common pattern: AfterWatermark.pastEndOfWindow() for the main firing, plus an early trigger like AfterProcessingTime.pastFirstElement().plusDelayOf(Duration.standardSeconds(30)) to give sub-minute visibility, and allow late firings (withAllowedLateness(...)) so late elements cause additional output panes. Use AccumulatingFiredPanes if you want updated aggregates (each late event updates the same logical result) or Discarding if you prefer only final outputs.

Handle very-late data by routing it to a side output / dead-letter/backfill topic instead of trying to update old windows indefinitely. For example, set allowedLateness to 30 minutes; events that arrive after that go to a "late" sink for offline backfill processing (a batch job that merges them into historical partitions). This prevents unbounded state growth and avoids blocking resources on extremely late records.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Tune watermark heuristics: ensure the source emits correct event timestamps and configure any watermark skew allowances in the Pub/Sub → Dataflow source. Monitor watermark lag (watermark - current event time) and late-event percentages. If watermark is consistently behind, adjust watermark estimation or increase worker pool to remove backpressure that delays watermark progress.

Finally, test with controlled late data to validate behavior: verify early results, subsequent updates when late data arrives, and that extremely late elements are handled by your side-path/backfill. Document SLAs (how soon final results are expected and how late data is treated) so downstream consumers know whether an aggregate may be updated later or not.

10. You need to build a near real-time reporting dashboard. How would you design Pub/Sub → Dataflow → BigQuery?

Start with Pub/Sub as the durable ingestion layer. Have producers publish well-formed messages with event timestamp and a producer-generated id (for dedupe). Partition topics logically (telemetry / events / alerts) so different consumers can scale independently and you can apply separate retention/quotas.

Process with a long-running Dataflow streaming pipeline that subscribes to Pub/Sub. Do minimal, fast transforms (parse, validate, enrich) and remove any heavy joins or expensive operations - keep the pipeline single-purpose so per-message latency is tiny. Use event-time semantics where needed, but prefer simple passthrough/field-extraction for dashboard data to avoid windowing delays.

Write into BigQuery using the low-latency path (Storage Write API or streaming inserts) so rows appear in the streaming buffer within seconds. Include insertId or event id for idempotency, write to a staging/streaming table if you need stronger control, and then MERGE into the final analytic table if dedup/upsert logic is required. Keep the BigQuery schema compact for dashboard fields to reduce ingestion cost and time.

Tune for low-latency reads: use a partitioned & clustered BigQuery table so dashboard queries scan minimal data. Consider using BigQuery BI Engine or cached materialized views for sub-second dashboard queries. Also design the dashboard queries to only read the most recent partition or window (e.g., last 5–15 minutes) to keep latency and cost down.

Operationalize with observability and SLOs: monitor end-to-end latency (publish → appear in BigQuery), Dataflow worker health, Pub/Sub backlog, and BigQuery streaming errors. Add alerts for increased lag, DLQ growth, or insert failures, and provide a simple replay mechanism (replay from Pub/Sub snapshots or GCS) so you can recover missed data without manual effort.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

11. Your Pub/Sub → BigQuery load is failing due to schema mismatch. How would you handle schema evolution?

Treat schema evolution as a planned process and automate checks before production loads. Maintain a canonical schema (Data Catalog / schema registry / versioned JSON) and have Dataflow validate incoming messages against it. If the incoming schema is compatible (additive nullable fields), let the transform coerce and pass records; if not, block and route to a DLQ for manual review.

Use a safe ingestion pattern: write raw/validated events into a staging table in BigQuery first, not directly to the production table. After staging, run a controlled MERGE or ETL job that enforces schema expectations and transforms fields as needed. This lets you handle type coercion, column renames and controlled backfills without corrupting production data or failing dashboard queries.

Adopt compatibility rules: allow only additive, nullable column additions automatically; require CI/CD migration scripts and a review process for renames, deletions, or type changes. Implement automated tests that run against a sample of incoming messages to surface incompatibilities before the change reaches production.

When a breaking change is required, deploy it in phases: (1) update producer to publish the new field while still sending the old one (dual-write), (2) update Dataflow to read the new field and write to staging, (3) run validation and backfill any missing values, and (4) flip the final table schema and remove the old column. Keep each phase reversible and document the window for cutover.

Finally, make the pipeline resilient to transient mismatches: capture mismatched rows to a DLQ with schema-diff metadata, alert owners, and provide tooling to reprocess corrected records. Keep auditable logs of schema changes and use versioned manifests for every load so you can replay or roll back if needed.

12. Business requires <10 seconds latency from ingestion to BigQuery. How would you achieve this?

Use a true streaming path: producers → Pub/Sub → long-running Dataflow streaming pipeline → BigQuery streaming (Storage Write API). Keep the pipeline continuous (don't start jobs per file) so there's no cluster startup delay. Keep transforms minimal and CPU-light to avoid per-message processing jitter.

Use the Storage Write API or BigQuery streaming inserts (with insertId) to get rows into BigQuery's streaming buffer within seconds. Avoid batch load jobs - they add minutes of latency. If your Dataflow workers may be preemptible, reserve a small pool of non-preemptible workers or set aggressive autoscaling so worker churn doesn't cause spikes in latency.

Co-locate resources regionally (Pub/Sub topic, Dataflow workers, and BigQuery dataset in the same region) to prevent cross-region network latency and egress. Tune client and pipeline batching knobs (publish batching and Dataflow sink batching) to small, low-latency thresholds so messages flow quickly while still benefiting from minimal batching efficiency.

Design queries and dashboard reads to hit recent streaming data efficiently - partition/cluster your table so queries scan tiny hot partitions, and enable BI Engine or cached materialized views for sub-second UI responsiveness. Also expose an incremental API or light-weight cache (Bigtable / Memorystore) for ultra-low-latency widgets if some panels must update in <1s while BigQuery handles the heavy aggregations.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, instrument rigorously and set SLOs: measure end-to-end latency percentiles, monitor Pub/Sub ack lag and Dataflow processing time, and alert on tail latency. Run load tests that mimic production burst patterns to tune worker counts, batching, and pipeline parallelism - meeting <10s consistently requires tuning for worst-case spikes, not just average load.

13. You must archive Pub/Sub messages as hourly Parquet files in GCS. How would you design this?

I'd use a long-running Dataflow streaming job as the main mover: subscribe to the Pub/Sub topic, parse messages and extract an event timestamp, then window by event-time into 1-hour fixed windows so files reflect the actual hour the events belong to. That preserves semantics when events are late or out of order.
Inside each window I'd key by whatever logical partition (e.g., topic/tenant) and use GroupIntoBatches / Combine.perKey or Beam's file sink to accumulate records and produce a small number of larger Parquet files per window rather than many tiny objects.
For output I'd write to a staging prefix first (gs://bucket/_tmp/.../hour=YYYY-MM-DD-HH/) and only finalize by atomically moving or renaming (or writing a _SUCCESS/manifest) into the final hourly folder (gs://bucket/archive/hour=YYYY-MM-DD-HH/). This prevents downstream readers from picking up partial files.
Include dedupe/ids: require producer event IDs and de-duplicate within a TTL using stateful RemoveDuplicates so retries/redeliveries don't create duplicate rows. Also include window id, shard id, and a content-hash in file names for traceability and idempotent replays.
Operationalize with monitoring and safety: emit metrics (files/hour, avg file size, late-event ratio), set alerts for high late-data or tiny-file spikes, grant least-privilege IAM to the pipeline, and keep an SLA for allowed lateness and retention so storage lifecycle rules can later archive older hours automatically.

14. A Pub/Sub → GCS pipeline is creating thousands of tiny files. How would you optimize file writing?

First, stop the immediate cause: increase the aggregation granularity - use larger windows (or batch multiple windows together) and write per-window files rather than per-message files. That single change often cuts file count dramatically.
Second, control sharding and batching in the sink: use withNumShards() or withMaxBytesPerFile() where supported, or GroupIntoBatches to build bigger bundles before writing. Aim for file sizes in the tens-to-hundreds of MB (not kilobytes).
Third, tune publisher and runner batching: enable client-side publish batching (max messages / max latency) and let Dataflow buffer slightly (small early firings for low latency but larger final writes) so you get both timeliness and big files.
Fourth, use a compaction step when you can't avoid tiny writes: run a periodic compactor (Dataflow batch or Dataproc Spark) that reads recent tiny files, merges them into compact Parquet files, writes to a compacted prefix, and publishes a manifest; then retire the tiny originals. Consider Hudi/Iceberg if you need continuous merges/upserts.
Finally, monitor and automate: track file count, avg file size, and compaction backlog; alert on tiny-file spikes; and make file-size targets part of your pipeline tests so the problem doesn't regress.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

15. Data in GCS must trigger downstream ETL after arriving from Pub/Sub. How would you implement this?

Have the ingestion path write final outputs atomically and then emit a reliable trigger: Dataflow (or the publisher) should finalize files into a .../ready/ prefix and then publish a small "ready" message to a Pub/Sub topic containing the partition/path and manifest (checksums, rowcount). That message is the canonical trigger for downstream ETL.
Downstream, run a Cloud Function / Cloud Run service or an Airflow (Composer) sensor that subscribes to the ready-topic, validates the manifest (checksums, _SUCCESS), and then starts the ETL DAG (via Airflow REST API or by publishing to a DAG trigger topic). This keeps the trigger event-driven and low-latency.
Ensure atomicity and idempotency: only publish the ready message after files are fully finalized and validated; include a unique run id in the message and record processed run ids in a control table so replays or duplicate triggers are ignored. Use a staging → promote pattern (write to _staging/ then move to /ready/) to guarantee consistency.
Handle errors and retries: if validation fails, route the ready event to a DLQ/quarantine with diagnostics and alert owners; ETL tasks should be idempotent so reprocessing is safe. If orchestrating from Airflow, have the DAG perform an extra validation step before heavy work and mark the run as skipped/failed cleanly when validation fails.
Monitor end-to-end: instrument publish → ready → ETL start latencies, manifest mismatches, and DLQ counts; keep runbooks for manual recovery (replay manifest, restore originals) so operators can quickly resolve problems without guessing.

16. You want to enrich Pub/Sub events with historical data in Spark. How would you build this pipeline?

Start by choosing where your historical data lives - BigQuery, GCS parquet, Bigtable/Redis for fast key-value lookups - because that determines join strategy. For small-to-medium reference tables (tens or low hundreds of MB), load them as broadcast (side) data into Spark executors so each streaming task can enrich events with an in-memory lookup. For large, frequently-updated reference sets, use a fast external key-value store (Bigtable/Redis) and do async/parallel lookups from the executors to avoid pulling huge state into memory.

Use Spark Structured Streaming as the runtime: read from Pub/Sub (via a connector or a lightweight bridge that writes Pub/Sub -> GCS then picks it up) as a streaming DataFrame, parse and normalize the event schema immediately, and extract a stable join key. If you broadcast a reference table, refresh the broadcast periodically (micro-batch boundary or on a detected change) so enrichment uses up-to-date history without restarting the job. If using an external store, batch lookups by key to amortize RPC cost and keep throughput high.

Design for correctness and failure modes: make enrichment idempotent (include event_id) and checkpoint streaming offsets to durable storage so restarts resume cleanly. If enrichment depends on slowly-changing attributes, store a version or timestamp in the enriched record so you know which snapshot was used for training or auditing. For late-arriving events, decide whether to enrich on arrival and also support late re-enrichment (replay) or keep a backfill process that re-enriches historical partitions.

Handle scale and hotspots by key-partitioning: if heavy skew occurs on certain keys, use client-side sharding (salt keys) or pre-aggregate / throttle those high-cardinality keys. Monitor enrichment latency, external-store latency, and executor memory. If external lookups become the bottleneck, introduce a caching layer (local LRU cache in executors or a shared in-memory cache like Memorystore) to reduce remote calls for hot keys.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, operationalize and test: run end-to-end load tests with realistic event rates and history sizes, add metrics (enrichment success rate, lookup latency, cache hit ratio), and build a replay plan (replay from Pub/Sub snapshots or GCS) so you can correct any enrichment bugs without losing data. Document the trade-offs (freshness vs memory vs latency) so stakeholders understand why a particular store or refresh cadence was chosen.

17. A team wants to run Spark Streaming jobs on Dataproc to process Pub/Sub data. How would you configure this?

Prefer Spark Structured Streaming on Dataproc because it gives nicer APIs and integrates with Dataproc autoscaling. Use a managed connector or a small bridge to feed Pub/Sub into Spark (for example a lightweight subscriber that writes rolling files to GCS which Spark reads, or a direct Pub/Sub connector if available in your image). Configure the job as long-running on an autoscaled ephemeral cluster sized for peak shuffle and state needs, and enable checkpointing to GCS so offsets and state survive restarts.

Right-size the cluster: pick VM types with good network and disk IO for shuffle, allocate executor cores/memory to match parallelism, and set spark.sql.shuffle.partitions to a sensible value based on cluster size. Use local SSDs for shuffle spill when jobs are shuffle-heavy. For cost control, use a small stable driver/primary pool of non-preemptible nodes and a larger worker pool of preemptibles for parallel stages, but ensure you have enough non-preemptible resources to maintain critical state and heartbeats.

Configure fault tolerance and state management: set Structured Streaming checkpoint directory and write-ahead logs to GCS, enable dynamic allocation or Dataproc autoscaling carefully, and tune spark.streaming.backpressure.enabled or similar flow-control knobs. For exactly-once semantics to sinks, prefer idempotent sinks or transactional table formats (Hudi/Iceberg) on GCS or use BigQuery's insertId when streaming into BigQuery.

Operationally, add logging and metrics (stream processing time, input rate, watermark lag, state size per key) to Cloud Monitoring. Provide a deployment pattern (packaged fat JAR with pinned Spark version, submitted via gcloud dataproc jobs submit spark or Dataproc Workflow Template) and CI to test on a sampled dataset. Finally, plan for maintenance: rolling restarts, image upgrades, and how broadcasted reference data will be refreshed without job downtime.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

18. Latency in Spark Streaming is too high compared to Dataflow. How would you decide which tool to use?

Compare requirements first: if you need low end-to-end, sub-second to a few-second latency, mature event-time/windowing semantics, and a fully managed autoscaling streaming runtime, Dataflow is usually the faster path out-of-the-box. If your workloads are batch-like, you already depend heavily on Spark libraries (MLlib, complex Spark SQL), or your team is Spark-expert and can tune micro-batches aggressively, Spark on Dataproc may be fine even if latency is higher.

Measure the actual bottleneck: is latency coming from ingestion (Pub/Sub publish/ack), windowing buffering, external lookups, or Spark micro-batch intervals and shuffle? Dataflow's streaming engine often handles event-time/watermarks and autoscaling with lower tail latencies; Spark Structured Streaming can reduce latency by shrinking trigger intervals, tuning small batch sizes, and optimizing resources, but at a cost of more operational tuning and possible increased overhead from many tiny tasks.

Consider semantics and features: Dataflow (Beam) has strong event-time semantics, flexible triggers, and built-in exactly-once patterns on certain sinks - ideal for late-event handling and correct event-time aggregates. Spark excels when you need heavy transformations, reuse of Spark ecosystem code, or when you want on-cluster in-memory operations for complex joins/ML. Choose the tool that minimizes risky rewrites and meets the correctness model you need.

Factor in ops and cost: Dataflow is serverless-managed - less Ops, predictable autoscaling, and often better tail latency without manual tuning. Spark/Dataproc gives more control and can be cheaper for steady, heavy compute loads but requires hands-on tuning (cluster sizing, shuffle tuning, GC). Also weigh team skills and deployment friction: if the team can quickly tune Spark to meet latency SLOs, that may win; otherwise Dataflow typically reaches low-latency SLOs faster.

Finally, prototype and quantify: implement a small, representative pipeline in both systems with your real data patterns, measure p50/p90/p99 latencies, operational effort, and cost under realistic bursts. Use that evidence to decide - the right choice is the one that actually meets your latency SLO with acceptable operational overhead.

19. A Pub/Sub message should trigger a Composer DAG to start ETL. How would you implement it?

I'd make the Pub/Sub → trigger path lightweight and reliable: have Pub/Sub deliver to a Cloud Function (or Cloud Run) that validates the event (bucket/prefix/manifest present, eventTime, run-id) and then triggers Composer. Triggering Composer is best done by calling the Airflow REST API to create a DAG run (authenticated with a short-lived service account token) or by publishing a well-formed "dag-trigger" message to a Pub/Sub topic that Composer's sensor DAG listens to. Using a small validation layer keeps Composer free of noisy, unauthenticated events and gives you a place to enforce contracts.

Make the trigger idempotent and include a unique run id in the payload. The Cloud Function should attach metadata (manifest path, checksum, producer id, generation) and store a control-row (BigQuery/Firestore) saying "trigger requested" before calling Airflow - that lets you reconcile duplicate function calls and prevents duplicate DAG runs. Use IAM so only the function's service account can create DAG runs or publish to the DAG-trigger topic.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

On the Composer side have a very small “trigger DAG” or sensor task that immediately re-validates (check _SUCCESS, manifest) and then kicks off the heavier ETL DAG via TriggerDagRunOperator or a direct operator call. Keep the first DAG task cheap so you can ACK the original Pub/Sub message quickly and let the heavy ETL run decoupled. If you prefer a single DAG, design the DAG to accept the run-id and path as templated params and bail early if the run-id is already recorded in the control table.

Add operational protections: use a Dead-Letter Topic for malformed or unauthorized events (Cloud Function pushes there), log every trigger attempt and response from Airflow, and emit metrics (triggers/sec, duplicates, trigger failures). Secure the REST call with workload identity or a signed token and rotate credentials via Secret Manager so Composer accepts only authenticated triggers. With this pattern you get an event-driven, auditable, and safe way to start ETL runs from Pub/Sub.

20. A Pub/Sub pipeline needs retries for failed events. How would you orchestrate this with Composer?

Assume Pub/Sub handles initial delivery retries; treat Composer as the orchestrator for business-level retries and backoff. Route transient failures in the consumer pipeline to a retry topic or a DLQ (configured on the Pub/Sub subscription or pushed by the Cloud Function). Then run a Composer DAG that periodically pulls messages from that retry/DLQ topic and performs controlled reprocessing attempts with exponential backoff logic implemented in the DAG (e.g., increasing delay tasks or Cloud Tasks with scheduled retry timestamps).

Keep retry state external and auditable: use a control table (BigQuery/Firestore) to track message_id, attempts, last_error, next_attempt_ts. The Composer DAG reads eligible rows (where next_attempt_ts <= now), attempts processing (or triggers the processing job), increments attempt count on failure, updates next_attempt_ts using an exponential backoff formula, and - if the max attempts are hit - marks the message as permanently failed and notifies owners. This prevents Composer from blindly re-scheduling too often and centralizes retry policy configuration.

For high-throughput reprocessing use batching in Composer: group many retry messages into a single reprocessing job (Dataproc batch or Dataflow batch) instead of invoking hundreds of small jobs. That makes retries more efficient and reduces orchestration churn. Also provide a manual retry UI or an Airflow operator to force immediate reprocess for priority items.

Always preserve idempotency in the processing itself - retries must be safe. Use dedupe keys / idempotency tokens so reprocessing a message does not create duplicate side effects. Finally, add monitoring and alerts for rising DLQ sizes and high retry rates, and provide a simple runbook for root-cause fixes (e.g., schema changes, downstream outage) so retries don’t just repeat a failing action indefinitely.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. You need a DAG to coordinate Pub/Sub ingestion, transformation, and loading into BigQuery. How would you design it?

I'd split the work into a clear, modular DAG with small, testable tasks: (1) *trigger / sensor* - wait for a Pub/Sub "ready" message or manifest and validate input; (2) *ingest to staging* - run a lightweight job that pulls Pub/Sub (or reads GCS staging files produced by Pub/Sub consumers) and writes normalized Parquet to a staging GCS path; (3) *transform* - run Dataproc/Dataflow batch that performs joins, cleans, and writes curated Parquet to a curated prefix; (4) *load to BigQuery staging* - use a BigQuery load job to move curated data into a staging table; (5) *quality checks & merge* - run SQL tests (row counts, nulls, schema) and then MERGE from staging into the production table; (6) *archive & notify* - move raw inputs to archive and emit completion metrics/alerts. Each step is a discrete operator so failures are isolated and retries are scoped.

Make each task idempotent and record metadata in a control table (run_id, input manifest, task status, checksums). Use staging tables and MERGE to guarantee idempotent loads into BigQuery instead of append-once semantics. Keep manifests and _SUCCESS markers so the DAG can validate completeness before the heavy transform step. If transforms are long, have the DAG submit an asynchronous Dataproc job and poll for completion rather than blocking a worker thread.

Parameterize the DAG (partition date, run id, source path) so it can handle scheduled runs and backfills. Enforce schema checks early: a small pre-transform validation task compares incoming schema to the canonical schema and fails fast to avoid wasted compute. For failures that are retryable, use task-level retry policies with exponential backoff; for poisoned data move the input to a quarantine location and notify owners.

Operationalize observability: emit detailed metrics per step (bytes processed, job duration, row counts), push logs to Cloud Logging, and set alerts for SLA breaches (e.g., transform > X minutes, load errors). Provide a manual "replay" path: the DAG should accept a list of input manifests to re-run a past ingestion deterministically. With modular tasks, idempotency, and strong validation gates, the DAG will be safe for production and straightforward to operate and audit.

22. A financial system requires strictly ordered transaction messages. How would you design this in Pub/Sub?

Start by using Pub/Sub ordering keys: have the producer set an orderingKey for every message that must be ordered together (for example accountId or ledgerPartition). Enable message ordering on the topic and subscription. Pub/Sub will guarantee delivery in publish order per ordering key and hold back later messages for that key until earlier messages are acked, so per-key FIFO is preserved without building your own sequencer.

Design your producer and consumer around the ordering trade-offs. If you need *total* global order, publish everything with the same single ordering key - but expect much lower throughput and a single hot partition. More commonly you choose a partition key (account, customer, region) so you get strong order per account while keeping throughput parallel. Document the ordering contract so everyone knows which key to use and why.

Make sure consumers process and ack strictly in-order per key. That means single-threaded processing for a given ordering key (or a keyed worker that enforces sequential execution), and you must **not ack** a message until it has been durably persisted/committed downstream. If a consumer crashes before acking, Pub/Sub will redeliver the same message and ordering will be preserved for that key once the message is processed and acked.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Protect against blocking: because Pub/Sub holds later messages for a key until earlier ones are acked, a stuck or poison message can halt progress for that key. Mitigate this with bounded retries and a Dead Letter Topic (DLQ) for repeatedly-failing messages - move the offending message to DLQ after max attempts so the key can continue. Also monitor per-key lag and blocked-key metrics so operators can act quickly on stuck partitions.

Finally, tune for performance and reliability: ensure ack deadlines and lease extensions are used if processing takes time, use autoscaling keyed workers to handle key-heavy workloads, and consider a pre-sharding strategy (salt the key) if a small set of keys becomes a hotspot. With ordering keys, careful consumer design, and DLQ protections you get predictable, per-key strict ordering suitable for financial flows.

23. You see duplicates in downstream BigQuery tables. How would you enforce idempotent processing?

Design for idempotency at the output: ensure each logical event carries a stable unique id (producer-generated event_id or deterministic composite key). When writing to BigQuery, surface that id as the dedupe key - either use insertId for streaming inserts (helps BigQuery de-duplicate short-term retries) or write to a staging table and perform a MERGE into the target table on the unique key so duplicate attempts do not create duplicate rows.

Keep a durable processed-key store for stronger dedupe guarantees. Before committing side-effects or writes, check-and-record the event_id in a transactional store (Cloud Spanner, Bigtable with conditional write, or Firestore with transaction). Only produce downstream writes when you can atomically mark the id as processed or when the write+mark is done in a single transaction or idempotent sequence. This prevents double-application when retries/redeliveries occur.

Make your pipeline idempotent end-to-end. Structure ops so you persist output **before** acking the Pub/Sub message (or only ack after a durable commit). If you cannot make the sink transactional with the dedupe store, write to a staging area and run a deduping MERGE/INSERT ... SELECT DISTINCT as a guaranteed idempotent step. For batch loads, include the event_id and filter by NOT EXISTS on the target key during the load/merge.

Instrument and audit: record every processed event_id, source timestamp and run id in a control table so you can reconcile publishes vs processed rows and re-run only missing events. Emit metrics for duplicate detections and set alerts when duplicate rates spike - that often points to producer bugs or incorrect ack semantics.

Finally, prefer defensive sink patterns: BigQuery MERGE on unique keys, inserting with insertId for streaming, or using table formats (Hudi/Iceberg) that support id-based upserts. Combining a unique id, a durable processed-key store or idempotent merge logic, and ack-after-durable-write will eliminate duplicates in downstream tables.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

24. Your consumer must process each message exactly once. How would you guarantee this?

Be explicit: true *exactly-once* end-to-end is hard; the practical approach is make processing observable and idempotent and use transactional writes/atomic checks so duplicate effects are prevented. The pattern is: (1) include a stable event_id in each message, (2) before applying side-effects, atomically check-and-set that id in a durable store, and (3) only if the check succeeds perform the side-effect - then ack the message. This gives logical exactly-once for the side-effect, even if Pub/Sub redelivers.

Use a transactional datastore to implement the check-and-set atomically. For example, use Cloud Spanner or a database that supports conditional inserts (INSERT IF NOT EXISTS) or transactions: attempt to insert a row keyed by event_id; if the insert succeeds, proceed with the work and commit; if it fails because the id exists, skip processing and ack. Alternatively, wrap the business write and the processed-id write in the same transaction when the sink supports transactions (Spanner, some SQL stores). This avoids races and guarantees a single application of effects.

If your sink is BigQuery (which doesn't support multi-row transactions), use an idempotent write pattern: write all events (with event_id) into a staging table, then run an atomic MERGE into the target using event_id as the key. Ensure you only ack Pub/Sub after the staging+merge is durable. For streaming sinks, leverage features like insertId (de-dup within BigQuery streaming buffer) but pair that with server-side dedupe for robustness.

Also consider at-least-once semantics plus compensating transactions when true transactional guarantees are impossible. For example, if you can't atomically check-and-apply, make your business operations undoable and run a reconciliation job that detects duplicates and corrects state. But prefer the deterministic check-and-set/transactional pattern wherever feasible for financial or critical systems.

Finally, implement monitoring and replayability: log every event_id, processing result, and timestamp; alert on duplicates or reconciliation mismatches; and provide a replay mechanism (re-publish events or run backfills) for rare recovery. With a producer-stable id, an atomic dedupe store (or transactional sink + merge), and ack-after-durable-commit, you get practical exactly-once guarantees for your consumer.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

25. Your Pub/Sub → Dataflow job is failing on certain malformed messages. How would you handle them?

First, never let malformed messages crash the pipeline - validate early. Add a parsing/validation ParDo as the first transform that attempts to parse each Pub/Sub message into your typed schema, and immediately routes successes to the normal pipeline and failures to a side output. Keep validation rules simple and deterministic (required fields, type checks, timestamp parse, JSON schema) so you catch obvious malformed payloads quickly.

Second, route malformed records to a durable dead-letter sink for inspection, not to an in-memory log. In Dataflow that means writing invalid messages (raw payload + error reason + metadata: message_id, publish_time, subscription, attempt count) to a DLQ topic in Pub/Sub or to a quarantined GCS prefix (with a manifest). Include enough context so an engineer can debug the producer or reprocess later without guessing what went wrong.

Third, instrument and alert. Emit metrics for validation failures (rate, error types, top offending producers) and create alerts for sudden spikes so you can act before DLQ piles up. Also expose a simple dashboard or log query that shows a sample of malformed messages and their common failure reasons - that speeds root-cause analysis and producer fixes.

Fourth, apply safe retry/backoff for transient problems. If validation fails for reasons that might be transient (temporary encoding, partial uploads), consider short automatic retries (with exponential backoff) before moving a message to DLQ - but guard this so you don't reprocess poison messages endlessly. Pub/Sub subscription retry settings and Dataflow's retry semantics can help coordinate this.

Finally, make reprocessing straightforward: keep DLQ records immutable and provide tooling (a replay script or an Airflow task) that can fix common issues (schema normalization, timezone coercion) and republish cleaned messages into the main topic or a staging topic for re-ingestion. Document the contract for producers so malformed messages fall dramatically over time.

26. A pipeline must route bad events into a DLQ for later reprocessing. How would you implement this?

Design a two-path pipeline: the normal path for valid events and a DLQ path for invalid/poison events. Implement validation as an explicit pipeline step (early ParDo) that returns either Ok(event) or Bad(event, reason). For Bad results, write the raw payload plus metadata (message_id, topic, producer id, error reason, timestamps) to a DLQ storage - use a Pub/Sub DLQ topic for message-driven reprocessing or GCS/BigQuery if you want durable searchable archives.

Choose DLQ storage by access pattern: use Pub/Sub DLQ topics if you plan automated replays (republish corrected messages easily), use GCS when messages may need manual inspection and large attachments, or use BigQuery when you want rich, SQL-based analysis of failure patterns. In all cases include a manifest/schema with each DLQ record so replays know what to fix and how to route back.

Add operational logic around DLQ handling: count attempts and move repeatedly failing messages to a "poison" sub-queue after N retries (preventing hot loops). Record attempt count and last error in a control table (Firestore/BigQuery) so your retry orchestrator knows which items to pick and which to escalate to on-call. Emit metrics and alerts for DLQ growth, and provide owners for top producers so issues don't linger.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Secure and make DLQ idempotent: include event_id and use dedupe logic when replaying so a re-ingested message doesn't duplicate effects. Tag DLQ records with remediation status (new, in-progress, fixed, ignored) and build simple tooling or an Airflow DAG to pick a batch of DLQ items, apply fixes or transformations, and republish into a staging topic for verification before routing to the main pipeline.

Finally, automate common fixes and replay flows. For predictable errors (e.g., date format, missing optional field), include automated normalize-and-republish scripts; for manual fixes, provide a small UI or notebook for analysts to inspect and correct sample messages. With clear routing, metadata, and tooling, DLQ handling becomes low-friction instead of an emergency-only process.

27. A backlog of DLQ messages must be replayed. How would you re-ingest them safely?

Start by analyzing the backlog: sample messages, cluster by error type, and estimate volumes and time-spread. Don't blindly replay everything - prioritize items by business value and fixability. Create a replay plan that separates automated fixes (mass-transform) from manual fixes (human review) so you don't waste operator time on easy corrections.

Next, prepare a safe replay path. Export DLQ items into a staging area (e.g., gs://staging/dlq-replay/run-<id>/) and run a transformation job that applies deterministic fixes (schema coercion, timezone corrections, null handling). Produce idempotent, cleaned messages that include original event_id and a replay_run_id in attributes. Publish those cleaned messages to a staging Pub/Sub topic that your pipeline treats as a "replay" input (or have a special DAG that consumes staging and writes to the normal processing topic after additional checks).

Enforce idempotency on ingest: ensure downstream sinks de-duplicate by event_id (BigQuery insertId or MERGE by key) or use a processed-key table so replays don't create duplicates. Only ack old DLQ messages in the control table when the replay run confirms a success. For large backlogs, process in bounded batches (e.g., 1k–10k messages) with checkpoints between batches so you can pause, inspect, and adjust without redoing everything.

Monitor and validate: after each replay batch, validate counts and sample outputs against expected results, monitor for new failures, and compare processed-event counts to the number of DLQ messages consumed. Emit metrics and alerts for replay failure rates so you can stop and debug before continuing. Keep detailed audit logs that map original DLQ record → replay run → final outcome for compliance and rollback.

Finally, document and close the loop with producers: publish a post-mortem listing root causes and fixes, improve producer-side validation if possible, and add automated guards (manifest checks, schema contracts) to prevent the same DLQ backlog from building again. Replaying DLQ safely is as much about automation and idempotency as it is about cleanup - treat it like a controlled migration, not a brute-force retry.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

28. IoT devices sometimes send events 30 minutes late. How would you configure your pipeline to handle them?

Treat timestamps as *event time* everywhere - extract the event timestamp from the device payload (don't rely on arrival time). Build your streaming job (Dataflow/Beam or Spark Structured Streaming) to window and aggregate by event time, not processing time, so late arrivals map into the correct windows when possible.

Set a sensible allowed lateness (e.g., 30–45 minutes) so the runner keeps window state open long enough to accept those delayed events. Combine that with triggers that emit early partial results and emit final/updated results after the watermark passes the end-of-window plus allowed lateness. Use accumulating panes if you want updated aggregates that incorporate late events.

Protect the pipeline from unbounded state growth by routing *very* late events (older than allowed lateness) to a side-output / DLQ for offline backfill. Have a batch job that periodically replays those very-late events into historical partitions (or runs a merge) so analytics remain correct without keeping streaming state forever.

Make deduplication explicit: producers should include stable event IDs and the streaming job should dedupe within the lateness window (stateful RemoveDuplicates or external dedupe store) so retries don't double-count when late messages arrive. Also monitor late-event ratios and watermark lag so you can tune allowed-lateness vs resource needs.

Finally, document the SLA for lateness (e.g., "aggregates are final after watermark + 30m") so consumers know whether to expect updates, and automate tests (inject late events) to validate the behavior before prod - that keeps expectations realistic and the system predictable.

29. Your aggregation queries in BigQuery are showing inconsistent results due to out-of-order events. How would you fix this?

Stop treating ingestion order as truth - re-aggregate on event time, not ingest time. Ensure your stream-to-BigQuery pipeline (Dataflow or other) tags each row with the event timestamp and writes that into the table; use that timestamp as the partition key or as the basis for downstream aggregation queries.

Prefer doing the windowed aggregation in the streaming engine (Dataflow) which understands watermarks and triggers, and write *finalized* aggregates or idempotent upserts into BigQuery. This way the engine can correctly include late/out-of-order events up to your allowed lateness and emit stable results for BigQuery to read, instead of letting BigQuery see raw out-of-order rows and produce transient inconsistencies.

If you must aggregate in BigQuery, stage incoming events first (staging table) and run controlled MERGE/batch aggregation jobs that process windows after a lateness buffer (e.g., run the daily/hourly rollup job at watermark + allowed lateness). That ensures the aggregation job sees all expected events for the window before computing results.

Add deduplication on event_id and idempotent merges so replays or retries don't double-count. Also consider using transactional table formats (Hudi/Iceberg) on GCS and then load compacted, ordered snapshots into BigQuery - this gives you cleaner, consistent inputs for analytic queries.

Finally, instrument visibility (watermark lag, late-event percentage, reorder rate) and publish a clear SLA: when aggregates are provisional vs final. That reduces confusion for downstream users and lets you tune lateness/latency trade-offs with data-driven evidence.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

30. Watermarks are advancing too quickly, causing late events to drop. How would you redesign?

First, identify why the watermark is too fast - often it's because the source uses processing-time heuristics or the watermark estimator assumes low skew. Fix the root: emit correct event timestamps from producers and ensure the source connector passes those timestamps to the runner so watermark estimation is meaningful.

Increase the watermark slack by configuring a larger allowed skew or allowed lateness in the runner (or adjust the watermark generator parameters). In Beam/Dataflow you can choose a different watermark estimator or tweak its delay so the watermark trails real event time by a safe margin for your observed out-of-order distribution.

If you can't afford a large global lateness (cost/state concerns), implement a hybrid strategy: keep a small allowed lateness for the main pipeline (fast results) and route late-but-acceptable events to a separate late-data path for replay/merge. Periodically run a backfill/merge job that incorporates those late events into historical windows or aggregates, producing final corrected outputs.

Consider using adaptive watermarking or custom watermark logic keyed by source partition - for example per-device or per-region watermarks with independent lag tolerances if skew varies widely across keys. This avoids one noisy producer dragging the global watermark and allows healthy partitions to progress faster.

Finally, monitor watermark metrics and late-event counters, and run chaos tests with synthetic out-of-order traffic so your watermark policy proves stable under real conditions. Document the final semantics (what "final" means, when corrections happen) so consumers know whether to read provisional or fully-final aggregates.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

31. Your consumer is lagging due to high Pub/Sub throughput. How would you scale it?

First, scale horizontally: add more consumer instances (more streaming workers or more pull clients). Pub/Sub parallelizes by having multiple subscribers pull from the same subscription; increasing the number of identical consumers immediately increases throughput. If you're running on Kubernetes/managed instance groups or Dataflow, increase replica count or worker pool size and let autoscaling handle elasticity for bursts.

Second, right-size instances and tuning knobs: use faster machine types or more vCPUs per worker if CPU is the bottleneck, and increase network/disk bandwidth for IO-heavy work. Tune client-side settings - increase maxOutstandingMessages / maxOutstandingBytes and batching parameters for publishes/ack extensions so each worker can process more messages in parallel without being I/O bound.

Third, use a more scalable runtime if needed: move from single-process consumers to a streaming runner (Dataflow, Flink) which handles parallelism, checkpointing and autoscaling better. These runners also implement efficient lease-extension and batching behavior out of the box so they often outperform ad-hoc thread pools when throughput grows very large.

Fourth, remove hotspots and work imbalance: partition hot keys (use additional ordering keys or sharding) so a single key doesn't throttle many workers, and push expensive work off the hot path (persist quickly and process heavy jobs asynchronously). If downstream sinks are the bottleneck, scale them independently (e.g., use BigQuery streaming + batching, or scale a DB tier) so consumers aren't blocked waiting for writes to finish.

Finally, instrument, test and iterate: measure per-worker throughput, CPU, memory, ack latency, and oldest-unacked age; run load tests to validate scaling behavior; and add autoscaling policies + alerts so you add capacity before SLAs slip. Document cost-vs-latency trade-offs so operations can pick sensible autoscale thresholds during spikes.

32. A backlog of millions of Pub/Sub messages has built up. How would you catch up processing?

First, diagnose the bottleneck: is the backlog due to slow consumers, insufficient workers, downstream write limits, or poison messages? Look at oldest-unacked age, redelivery counts and downstream error rates - that tells you whether to add consumers, speed up processing, or fix data issues before simply throwing resources at it.

Second, scale up aggressively but safely: temporarily increase consumer parallelism and worker size, or spin up a dedicated "catch-up" fleet (Dataflow workers or ephemeral pods) to drain the backlog. Use autoscaling policies that let you burst beyond normal steady-state capacity, then scale back when the backlog is cleared to avoid runaway cost.

Third, process in prioritized batches: separate high-value / recent messages from low-priority ones and catch up important partitions first. If messages are grouped by key, process backlog keys in parallel by sharding or creating multiple subscriptions for partitioned workloads (fanout from the original topic to multiple subscriptions if appropriate) so you can allocate workers to hot partitions without impacting normal traffic.

Fourth, protect downstream systems and use backpressure: increase sink throughput (bulk writes, larger batch sizes), or write backlog outputs to intermediate durable storage (GCS/BigQuery staging) so you don't overload databases with many small writes. If some messages are causing repeated failures, move them to a DLQ so they don't keep throttling catch-up progress.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, validate and automate replay: if you need to replay older messages intentionally, use Pub/Sub snapshot/seek (or re-publish from stored raw files) in controlled windows, track progress with a control table, and monitor success metrics. After catch-up, analyze root causes (scaling limits, hot keys, or processing inefficiency) and implement long-term mitigations like autoscaling, batching, and better partitioning.

33. You must control subscriber parallelism to avoid overloading a downstream system. How would you configure this?

Start by applying flow-control at the consumer: limit maxOutstandingMessages and maxOutstandingBytes (or equivalent) in the Pub/Sub client so a single subscriber only pulls a bounded set of messages. That prevents a consumer from absorbing more messages than it can safely persist downstream and gives predictable concurrency per instance.

Second, limit the number of parallel consumers and size each worker appropriately. Use a fixed replica count or an autoscaler with an upper cap so the total parallelism (workers × messages-per-worker) stays below the downstream capacity. If using Dataflow, configure the worker pool size and maxNumWorkers; if using Kubernetes or VMs, cap replica count in the deployment/instance group.

Third, add rate-limiting or token-bucket logic around the downstream calls: each worker should gate calls to the downstream service with a shared rate limiter (Redis, token-bucket service, or a local leaky-bucket) so aggregate throughput never exceeds the target. This is particularly useful when many consumers share one downstream endpoint.

Fourth, add backpressure-friendly architecture: instead of direct synchronous calls to the downstream system, write processed payloads to a buffer (BigQuery staging, GCS, or a buffering queue) and let a controlled set of writer processes drain the buffer at the acceptable rate. This decouples ingestion parallelism from sink concurrency and lets you scale ingestion without overloading the sink.

Finally, monitor and enforce via automation: emit metrics for downstream queue length, error rate and latency, and use an autoscaler that respects these signals (scale down consumers when downstream latency spikes). Maintain a documented capacity budget (max requests/sec) and implement circuit-breaker behavior so the system degrades gracefully rather than collapsing under overload.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

34. A downstream job failed, and you need to replay last 24 hours of messages.

First, pick the replay source. If you have Pub/Sub message retention enabled (or snapshots) you can replay directly from Pub/Sub by seeking a subscription to the timestamp 24 hours ago. If you don't have retention long enough, replay from a durable backup (recommended): a copy of raw messages kept in GCS (or BigQuery) by your ingestion pipeline. Always design ingestion to write a raw copy for exactly this use-case.

Second, prepare the environment: stop or pause the downstream consumer(s) or point them to a temporary staging subscription so you don't double-process live traffic. Create a new subscription for replay (clean slate) so the live pipeline is unaffected, or use the existing subscription but only after you've snapshot/paused production consumers. This isolates the replay and makes rollbacks safer.

Third, perform the actual replay. Option A: use Pub/Sub Seek to timestamp (seek the replay subscription to T = now − 24h) so retained messages will redeliver. Option B: if using GCS backups, run a replay job that reads the raw files for the 24h window and republishes messages (preserving original event_id and attributes) to a staging Pub/Sub topic that your replay consumers read. Whichever you choose, publish with the original event id so downstream dedupe can work.

Fourth, run the replay at controlled scale: throttle or shard the replay (by time-range or key) to avoid overwhelming downstream systems. Monitor processing rates, error counts, and duplicate-detection metrics. Use autoscaling for replay workers but cap concurrency to keep downstream systems healthy; if the downstream system is fragile, replay into a staging sink and batch-load rather than live-writing.

Finally, validate and finalize: compare counts/checksums between original publish and processed outputs, run data quality tests, and only after verification promote replay results to production (or switch consumers back). Record the replay run-id in your control table and keep audit logs. After success, resume normal flows and document root cause and preventive fixes so future replays are less likely to be needed.

35. Your compliance team requires messages retained for 7 days. How would you configure Pub/Sub?

Enable and set message retention on the topic/subscription so Pub/Sub keeps messages (even after ack) for 7 days. In Pub/Sub this is done by configuring the topic's message retention duration (max 7 days is commonly used); with that set, acknowledged messages remain available for seek and compliance needs. Also consider using snapshots to capture an immutable point-in-time view when you need stricter auditability.

Next, persist raw copies outside Pub/Sub as an extra compliance guarantee - write every incoming message (or its copy) to immutable storage like GCS (with object versioning and CMEK if required) or to BigQuery. Pub/Sub retention is useful, but durable archival in GCS gives you easier long-term access, legal hold, and richer search/forensics tools than relying only on broker retention.

Then, lock down access and retention policies: use IAM to limit who can modify topic retention or delete snapshots, enable audit logs, and apply bucket-level retention/holds in GCS if you store copies. If legal requirements demand immutable retention, use bucket retention policies and disable object deletes for the required period rather than depending solely on Pub/Sub settings.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Also document operational limits and costs: longer retention increases storage usage and potentially billable operations; communicate this to compliance so they understand trade-offs. Finally, test the retention and retrieval process (seek to an older timestamp, restore from GCS archive) so you can reliably demonstrate compliance when auditors ask.

36. A bug in transformation logic corrupted data. How would you use replay to reprocess messages?

First, stop the bleeding: halt the pipeline that applies the bad transformation (or switch it to a safe mode) so new data isn't further corrupted. Identify the exact time window(s) and job runs affected by the bug and record the original raw inputs for those periods (from Pub/Sub retention or your raw GCS archive).

Second, fix the transformation logic in a branch and add tests that specifically catch the corruption case. Avoid patching in-place on production - create a tested, versioned artifact (Dataflow template, Spark JAR) that contains the corrected logic and logging to confirm the fix's behavior.

Third, reprocess the affected messages using a safe replay path: either Seek the subscription back to the earliest corrupted timestamp (if retention/snapshots allow) or republish the raw messages from the GCS archive into a staging Pub/Sub topic. Run the corrected transformation against that staging topic and write outputs to a staging dataset (not directly over the production tables).

Fourth, validate thoroughly: compare the newly produced outputs against golden checks (row counts, keys, checksums, business-level validators). Use a MERGE/replace pattern to update production only after successful validation - e.g., write corrected rows to a staging BigQuery table and then MERGE into production using dedupe keys, or swap partition files atomically if using GCS/Parquet. This avoids partial overwrites and preserves auditability.

Finally, document and automate prevention: add pre-deploy tests (sample replay), enable message-level checks earlier in the pipeline, keep raw archives immutable, and record the replay run in your control table. If possible, add feature toggles or canary runs for future transformations so you can catch logic regressions before they touch production at scale.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

37. Invalid JSON messages are breaking downstream jobs. How would you validate and filter them?

Start at ingestion with a lightweight validation gate: parse each incoming Pub/Sub message as the very first step in your pipeline (Cloud Function, Dataflow ParDo, or a small pre-processor). If the parser fails, immediately route the raw payload to a side path instead of letting it flow into main transforms - this prevents a single bad message from crashing downstream logic.

Use a formal JSON schema (JSON Schema / Avro / Proto) that producers and consumers agree on. Validate against that schema during ingestion so you catch structural problems (missing required fields, wrong types) early. Keep the validator deterministic and strict for production-critical fields, but allow non-blocking warnings for optional or evolving fields so you don't break on benign differences.

For malformed messages, write them to a durable DLQ/quarantine (Pub/Sub dead-letter topic or GCS bucket) with rich metadata: original message bytes, message_id, publish_time, subscription, validation error, checksum and any producer id. That preserves context for debugging and prevents retries from looping on poison data. Don't delete the raw corrupted object - keep it immutable for audits and replay attempts.

Alert and instrument: emit metrics (validation failure rate, top error types, offending producer ids) and create an alert when failure rate spikes. Provide a small diagnostics pipeline or UI that groups errors by type and shows example payloads so producers can quickly fix payload generation issues. Automate common fixes where safe (e.g., minor timestamp format fixes) but require manual review for unclear corruptions.

Finally, make reprocessing easy: retain malformed messages long enough and provide tooling (Airflow task or a replay script) to re-run cleaned messages into the main topic once fixed. Document the contract for producers (schema, encoding, size limits), add CI tests for schema changes, and keep the validation logic in a shared library so both producers and consumers validate consistently.

38. Schema changes in Pub/Sub events are causing failures. How would you design schema evolution handling?

Adopt a schema registry and versioning policy up front (JSON Schema / Avro / Protobuf + central registry). Every change must have a declared compatibility level (backward, forward, or none). Producers should register new schema versions; consumers should check compatibility before accepting changes so surprises are avoided.

Enforce producer-side validation and staged rollout: producers should publish new fields in an additive, nullable way first (dual-write where possible), and include a schema_version attribute in messages. Consumers should tolerate unknown fields (ignore extras) and use schema-version-aware deserializers. Breaking changes (renames, type narrowing, deletions) must go through a migration plan: dual-write, consumer updates, and then safe removal.

Use a staging flow for schema changes: route new-schema messages to a staging topic or flag them with schema_version so a validation pipeline can exercise downstream consumers (smoke tests) before the change goes to production. Automate compatibility checks in CI-validate that new schema versions are compatible with consumer expectations using the registry rules.

When loading into sinks like BigQuery, use a staging table first. Validate and coerce types there; run MERGE or controlled schema-migration SQL rather than blindly loading. Keep migrations reversible (write new columns, backfill if needed, then switch readers) and record migration runs in a control table so rollbacks are traceable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, communicate and govern: publish schema change notices, require owners to sign off on breaking changes, monitor schema-drift metrics, and keep tooling for automated reprocessing/backfill when incompatible changes slip through. This mix of registry, compatibility rules, staged rollout and automation prevents most runtime failures from schema evolution.

39. You need to log all malformed messages for audits. How would you build this pipeline?

Capture malformed messages at the validation gate and persist them immutably with rich metadata. The easiest robust pattern is: validate → on failure write a record (raw payload, error reason, message_id, publish_time, subscription, checksum, producer id, validation timestamp) to a partitioned, append-only audit store - either GCS (date-partitioned objects) for binary blobs or BigQuery for queryable structured logs.

If you need long-term, tamper-evident storage for compliance, write the raw corrupted payloads to GCS with bucket retention/holds and object versioning enabled; store the metadata (pointers, checksums, run id) in a BigQuery audit table so auditors can query frequency, producers, and failure reasons. Keep the GCS blobs immutable and restrict delete permissions via IAM to satisfy compliance requirements.

Make sure the logging is atomic and reliable: do the DLQ/GCS write before acking the Pub/Sub message (or as part of the same transactional flow where possible) so you never lose a malformed message. Tag each audit record with a replay status (new, investigated, corrected, replayed) and keep tooling (Airflow task or a small UI/notebook) to inspect, mark, correct and optionally republish cleaned messages.

Add observability and retention controls: partition audit logs by date for efficient queries, enforce lifecycle rules for archival or extended retention, and expose dashboards for audit teams showing counts, top error types, and producer trends. Finally, secure access tightly (least privilege), log who inspected/modified audit entries, and include the audit pipeline in your DR and backup plans so audit data is preserved according to policy.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

40. You need to build a fraud detection system with sub-second latency. How would you design it using Pub/Sub?

Start with Pub/Sub as the ingestion front door: devices/apps publish every event (transaction, auth attempt) into a dedicated topic. Require a small, strict schema and a producer-generated id so every event is traceable and dedupable. Use short-lived credentials or an edge gateway for constrained clients so security and throttling are enforced close to the source.

Put a low-latency stream processor (Dataflow streaming or Flink on GKE) subscribed to the topic to evaluate fraud rules in-memory. Keep the pipeline minimal and CPU-light per-event: do fast stateless checks (velocity, blacklists) and if necessary query a fast online store (Bigtable/Redis) for stateful checks (historical counts, recent device fingerprints). Cache hot keys in-process and update the online store asynchronously to avoid blocking per-event decisions.

For more complex models, use a two-tier approach: the fast path is deterministic rules and a lightweight ML model (small, optimized, possibly served via an in-process library or TensorFlow Lite) that returns a score <10ms. The slow path sends suspicious events to an async scorer or batch ML service and flags events as "review" until the deeper result arrives. Always emit the quick decision to downstream systems immediately (block / allow / risk-score).

Push decisions and telemetry to downstream sinks: immediate actions go to a control topic (for blocking, session termination, or real-time alerts) and to a low-latency store (Bigtable/Redis) for lookups; send detailed event + features to a durable sink (BigQuery/GCS) for offline model training and audits. Ensure idempotency on action side-effects - write the action and a processed-event-id atomically or record the action in a transactional store before acknowledging the Pub/Sub message.

Finally, operationalize aggressively: monitor tail latencies, model drift, false positive rates and queue backlogs. Add a human-review workflow (review queue, feedback loop) so analysts can mark false positives and that feedback feeds model retraining. Keep runbooks for fallback: if the real-time pipeline fails, degrade to safe-mode rules or redirect to an external blocking service so fraud protection remains continuous.

41. Marketing wants a clickstream dashboard refreshed every few seconds. How would you build Pub/Sub → Dataflow → BigQuery → Looker?

Ingest click events into Pub/Sub with small messages and consistent attributes (user_id, session_id, page, event_time). Enforce a schema and include a producer event id to help dedupe. Partition topics logically by event type if different consumers will scale differently.

Run a long-running Dataflow streaming job that subscribes to the topic, does light enrichment (geo, user segments via a cached lookup), removes duplicates in a short TTL window, and computes very short windows (5–10s) for near-real-time aggregates. For final analytic accuracy you can also produce longer windows (1–5min) as updated panes; write both micro-aggregates and finalized aggregates so dashboards can show instant and stable views.

Write micro-aggregates directly into BigQuery using the Storage Write API or streaming inserts with insertId for idempotency. Use partitioned & clustered tables optimized for recent time ranges - materialize a small "hot" table for the last few minutes and a separate historical table for older data. Alternatively keep micro-aggregates in a low-latency cache (Bigtable or Redis) for the most frequent dashboard panels and sync aggregated snapshots to BigQuery for Looker to query.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

In Looker, build dashboards that query the hot partitions or use a derived table that reads only the last N minutes. Enable caching/BI Engine for the most accessed tiles so sub-second dashboard refresh is feasible. Design the dashboard queries to request only the newest partitions and keep heavy joins out of the hot path to avoid latency spikes.

Instrument end-to-end: measure publish → appear-in-BigQuery latency (p50/p90/p99), Dataflow processing time, and query times. Set SLOs (e.g., 5s p95) and alerts for backpressure. Finally, tune costs by keeping micro-aggregate retention short in BigQuery (or use cache), compact streaming writes into periodic snapshots, and rely on the streaming pipeline for freshness while letting scheduled batch jobs recompute canonical aggregates if needed.

42. An IoT use case requires anomaly detection on millions of sensor readings. How would you implement this pipeline?

Ingest sensor telemetry into Pub/Sub (or Pub/Sub Lite for very high throughput) and include event_time, device_id, and schema-version. Use edge filtering/gateway aggregation for ultra-high-volume sensors so you reduce noisy chatter and add device metadata close to the source.

Use a streaming processor (Dataflow or Flink) that applies lightweight per-event checks (thresholds, simple z-score) and maintains short-term per-entity state (sliding-window stats: mean, variance, recent max/min) in runner state or a fast external store. For large state (many devices) keep state in an external low-latency store (Bigtable) and use local caching to avoid per-event RPCs for hot keys. Emit immediate anomaly alerts for obvious violations.

Parallel that with a feature-extraction stream that computes time-windowed features (rolling windows, counts, frequency-domain summaries) and writes feature vectors to a training store (BigQuery or GCS) for periodic model training. Train ML models (isolation forest, autoencoder, or lightweight neural nets) offline on this data and deploy the model into an online scorer: either embedded in the streaming job or served from a model server with very low latency (Vertex AI Prediction with low-latency endpoint, or an in-memory library for speed).

For complex anomalies that require aggregation across many devices or long history, run a near-real-time batch job (micro-batch Dataproc) that recomputes heavy features and runs a more sophisticated model, flagging eventual anomalies and enriching the streaming results. Store all raw events and detections in durable storage (GCS/BigQuery) for audit, training and explainability.

Ensure operational readiness: tune window sizes and model thresholds to balance false positives/negatives, add a feedback loop so human analysts can label data and improve models, and implement backpressure and DLQ for malformed input. Monitor detection latency, throughput, and model drift; periodically retrain models with the latest labeled data and re-deploy with A/B testing to verify improved detection.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

43. Your company is moving from Kafka to Pub/Sub. How would you design a migration strategy?

Start with a careful inventory and risk map: list topics, throughput, retention, consumers, producer libraries, ordering needs, schema usage (Avro/Proto), ACLs and any features you rely on (compaction, log retention, exactly-once semantics). That tells you which Kafka features you must replicate in Pub/Sub (ordering keys, DLQs, retention) and which consumers or producers need code changes. Also measure traffic patterns and storage so you can size Pub/Sub quotas and downstream systems during the cutover.

Build a compatibility layer and plan for dual-run testing. Create a connector/bridge that mirrors Kafka topics into Pub/Sub (or vice-versa) so you can run consumers against Pub/Sub without touching producers. For producers that must change, prefer adapters or client wrappers that can publish to both systems. Keep message schemas in a registry (or Data Catalog) and ensure message IDs/attributes are preserved so dedup and replay are possible during the transition.

Design a staged cutover: start with non-critical topics, run consumers against Pub/Sub mirrored data, validate end-to-end behavior and performance, then move important topics. For each topic, verify ordering semantics (use ordering keys in Pub/Sub), retention, DLQ behavior, and downstream idempotency. Keep a rollback plan: if something breaks, revert consumers to Kafka or resume mirroring while you fix the issue.

Operationalize the migration: automate the bridge, add health checks, and create a control table that records migration state per topic/partition (mirrored, tested, cutover). Train teams on new operational primitives (Pub/Sub quotas, snapshots, seeking) and update runbooks for monitoring, alerting, and incident response. Run load tests and chaos tests (consumer restarts, network faults) so you've proven reliability at scale.

Finally, decommission carefully: after successful cutover and a stabilization period, stop mirroring and retire Kafka topics incrementally. Archive Kafka offsets and metadata for compliance. Capture lessons learned, update diagrams and runbooks, and finalize team ownership so the new Pub/Sub-based platform is maintainable long term.

44. A hybrid system requires both Kafka and Pub/Sub during transition. How would you implement this?

Treat the hybrid period as a well-controlled bridge: deploy a bi-directional or one-way mirroring layer that reliably copies messages between Kafka and Pub/Sub. Use a connector or lightweight service that reads from Kafka and publishes to Pub/Sub (preserving keys, timestamps, and a producer event_id attribute), and vice-versa if some consumers still read from Kafka. Make the bridge idempotent and observable - log mapping decisions and failures to a control stream for operators.

Use consistent schemas and metadata across systems. Keep a central schema registry and ensure each mirrored message carries a schema_version and origin attribute so downstream logic can detect source and apply appropriate parsing. Also preserve a stable unique id in the payload so deduplication on the consumer side works regardless of whether messages came directly from Kafka, Pub/Sub, or the bridge.

Run consumers in a dual-read mode when possible: have critical consumers subscribe to Pub/Sub and also read from Kafka (or read only Pub/Sub while Kafka is mirrored) during testing windows to compare outputs. Where dual-read isn't possible, enforce strict testing: run shadow consumers that process mirrored messages and compare results with production runs to surface discrepancies before switching live traffic.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Manage ordering and throughput trade-offs: if you need strong per-key ordering, map Kafka partition key to Pub/Sub orderingKey; be aware that Pub/Sub ordering is per-key and blocks on unacked messages so plan capacity and DLQs accordingly. For high-throughput topics, shard mirroring processes and throttle re-publishes to avoid overloading target systems; use batching and resumable checkpoints to make mirroring robust.

Finally, have a clear cutover plan and governance: pick a date or success criteria per topic, communicate to producers and consumers, and automate the switch using infrastructure code. Keep the bridge in place for a defined stabilization window for observability, then retire the Kafka-facing components as teams confirm correctness. Throughout, document assumptions and update runbooks so the hybrid stage is safe and reversible.

45. You are tasked with modernizing a batch ETL to an event-driven Pub/Sub pipeline. How would you approach it?

Begin by mapping the batch ETL's inputs, transformations, and outputs to events and streaming semantics. Identify the events that matter (ingest, update, delete), the idempotency keys, and which parts of the batch logic are naturally incremental versus those that require full-window context (joins, heavy aggregations). That tells you what you can do in-stream and what still needs micro-batches or periodic batch jobs.

Design an event model and topics: define small, well-typed messages (use Avro/Proto/JSON schemas), assign ownership, and create topics for raw events and for curated/derived streams. Introduce a schema registry and require producers to attach event_id, event_time, and schema_version. Start by streaming the raw events into Pub/Sub, with a reliable archive to GCS so you keep a backup for replays and audits.

Re-implement transformations as a mix of streaming operations and periodic jobs: use Dataflow (or Flink) for low-latency enrichments, simple joins and windowed aggregates; keep heavy, global joins or historical backfills as scheduled micro-batch jobs that consume from a durable store or from compacted streams (compact/merge sinks like Hudi/Iceberg on GCS). Make each streaming transformation idempotent and checkpoint-aware; write curated outputs in partitioned Parquet and also materialize key aggregates into BigQuery for analytics.

Plan cutover and backfill: run the new streaming path in parallel with the batch ETL for a while, compare outputs, and perform a one-time backfill to seed historical state for streaming consumers. Use a control table to track processed event ids and a replay mechanism for missed events. Also create a graceful fallback so if streaming misbehaves you can temporarily run the batch ETL again while you fix issues.

Finally, operationalize and govern: add monitoring for publish/consume rates, late-event ratios, DLQ growth and end-to-end correctness metrics; add alerts and runbooks. Train teams on new patterns (schema evolution, stream debugging), and iterate - modernization is incremental: start by streaming easily-decomposable steps, prove savings/latency benefits, then migrate more complex pieces once your platform and team maturity grow.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar